STEP 1 : Open Google Colab

STEP 2 — Import Libraries

```
# Numerical operations
import numpy as np

# Data handling
import pandas as pd

# Text preprocessing tools
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Dataset loading
from tensorflow.keras.datasets import imdb

# Model building
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, Dropo

# Train-test split
from sklearn.model_selection import train_test_split

# Evaluation metrics
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, co

# For visualization (optional)
import matplotlib.pyplot as plt
```

STEP 3 — Load and Explore Dataset

```
# Load IMDB dataset (top 20,000 words)
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=20000)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("Example review (encoded):", X_train[0])
print("Label:", y_train[0])  # 0 = negative, 1 = positive
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training samples: 25000
Testing samples: 25000
Example review (encoded): [1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66,
Label: 1
```

STEP 4 — Text Preprocessing

```
# Define maximum length
max_length = 200

# Pad sequences so all reviews have equal length
X_train = pad_sequences(X_train, maxlen=max_length, padding='post')
X_test = pad_sequences(X_test, maxlen=max_length, padding='post')

print("Shape after padding:", X_train.shape)
```

```
Shape after padding: (25000, 200)
```

STEP 5 — Vocabulary and Embedding Preparation

```
# Get word index from IMDB dataset
word_index = imdb.get_word_index()

vocab_size = 20000
embedding_dim = 100
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
# Load pretrained GloVe embeddings
embeddings_index = {}

# Download GloVe embeddings if not already present
import os
if not os.path.exists("glove.6B.100d.txt"):
    print("Downloading GloVe embeddings...")
    !wget http://nlp.stanford.edu/data/glove.6B.zip
    !unzip -q glove.6B.zip
    print("GloVe embeddings downloaded and unzipped.")

with open("glove.6B.100d.txt", encoding="utf8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        vector = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = vector

print("Total GloVe words loaded:", len(embeddings_index))
```

```
Downloading GloVe embeddings...
--2026-02-19 10:08:25--  http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
```

```
--2026-02-19 10:08:25--  https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-02-19 10:08:25--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: ‘glove.6B.zip’

glove.6B.zip        100%[===================>] 822.24M  4.92MB/s    in 2m 43s

2026-02-19 10:11:08 (5.05 MB/s) - ‘glove.6B.zip’ saved [862182613/862182613]

GloVe embeddings downloaded and unzipped.
Total GloVe words loaded: 400000
```

Create Embedding Matrix

```
embedding_matrix = np.zeros((vocab_size, embedding_dim))

for word, i in word_index.items():
    if i < vocab_size:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector
```

STEP 6 — Train–Test Split

```
# Further split training into validation set
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42)
```

STEP 7 — Build 1D CNN Model with Pretrained Embeddings

```
model = Sequential()

# Embedding layer with pretrained weights
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=max_length,
                    trainable=False))  # Freeze embeddings

# 1D Convolution layer
model.add(Conv1D(filters=128, kernel_size=5, activation='relu'))

# Global Max Pooling
```

```
model.add(GlobalMaxPooling1D())

# Fully connected layer
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))

# Output layer
model.add(Dense(1, activation='sigmoid'))

# Compile model
model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarn
  warnings.warn(
```

Model: "sequential"

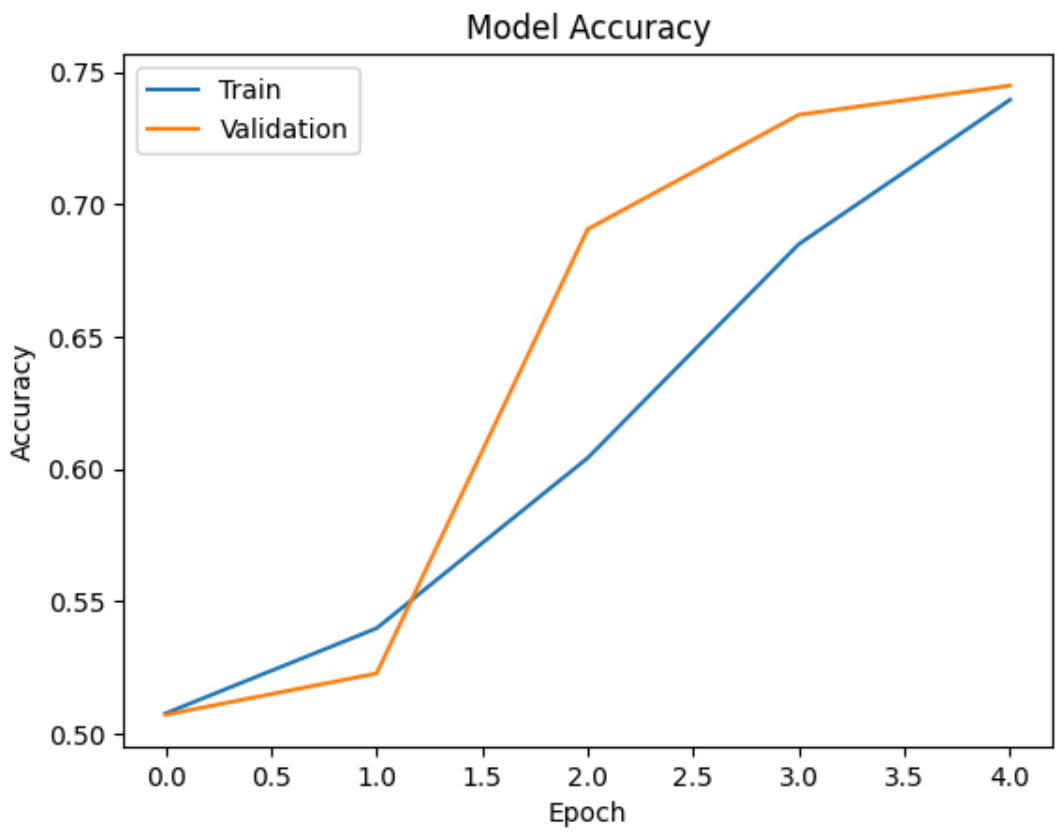
Layer (type)	Output Shape	Param #
embedding (Embedding)	?	2,000,000
conv1d (Conv1D)	?	0 (unbuilt)
global_max_pooling1d (GlobalMaxPooling1D)	?	0
dense (Dense)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

Total params: 2,000,000 (7.63 MB)
Trainable params: 0 (0.00 B)
Non-trainable params: 2,000,000 (7.63 MB)

STEP 8 — Model Training

```
history = model.fit(X_train, y_train,
                    epochs=5,
                    batch_size=128,
                    validation_data=(X_val, y_val))
```

```
Epoch 1/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 31s 188ms/step - accuracy: 0.5029 - loss: 0.7088 - val_acc
Epoch 2/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 34s 215ms/step - accuracy: 0.5314 - loss: 0.6892 - val_acc
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 37s 189ms/step - accuracy: 0.5895 - loss: 0.6709 - val_acc
Epoch 4/5
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 42s 193ms/step - accuracy: 0.6702 - loss: 0.6116 - val_acc
Epoch 5/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 39s 183ms/step - accuracy: 0.7381 - loss: 0.5347 - val_acc
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()
```